19. Profiling, Race Detector, Performance Mental Model

19. Profiling, Race Detector, Performance Mental Model

Phase: Phase 3 — Idioms and engineering judgment
Estimated time: 1–2 days
Slug: `19-profiling-performance`

Learning objectives

- Run the race detector and interpret failures
- Capture CPU and memory profiles with pprof
- Reason about allocations and escape analysis at a high level
- Optimize only with evidence

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Race detector

```
go test -race ./...
```

Detects concurrent reads/writes without synchronization. **Use it in CI.**

pprof

```
go test -bench=. -cpuprofile=cpu.out
go tool pprof cpu.out
```

Or expose `/debug/pprof` on a debug port in services (carefully locked down).

Mental model

1. Correctness first
2. Measure
3. Fix algorithmic issues before micro-opts
4. Reduce allocations in hot loops (preallocate slices, avoid string concat in loops)

Escape analysis decides heap vs stack; pointers and interfaces often force heap allocation.

Common mistakes (especially from Python instincts)

- Optimizing without benchmarks
- Disabling race detector because “it’s slow” in CI forever
- Caching everything without eviction strategy

Exercises

Work in `exercises/19-profiling-performance/`. Read that folder’s `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.

- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/19-profiling-performance/` until you have an honest failing attempt.

Self-check

1. What does -race detect?
2. Name two profile types
3. Why measure before optimizing?

Further reading

- https://go.dev/blog/pprof
- https://go.dev/doc/articles/race_detector